

Report of Workflow

Feature Selection & Model Creation

Table of Contents

- Step 1 — Imports & Library Setup
- Step 2 — Loading the Cleaned Dataset
- Step 3 — Encoding Categorical Features
- Step 4 — Feature Scaling with StandardScaler
- Step 5 — Establishing a Baseline with DummyClassifier
- Step 6 — Detecting Multicollinearity
- Step 7 — Measuring Feature Importance
- Step 8 — Dropping the Redundant Feature
- Step 9 — Recursive Feature Elimination with Cross-Validation (RFECV)
- Step 10 — Preparing the Final Feature Matrices
- Step 11 — Multi-Model Performance Benchmarking
- Step 12 — Hyperparameter Tuning with GridSearchCV
- Step 13 — Final Model Evaluation & Diagnostic Plots
- Step 14 — Clinical Feature Importance Ranking
- Step 15 — Saving Production Assets
- Step 16 — Pre-Deployment Safety Audit

STEP 1

Imports & Library Setup

In these steps, we are importing the necessary libraries.

STEP 2

Loading the Cleaned Dataset

In this step, we are importing the pre-processed dataset, which has had null values and outliers handled, and has been cleaned.

STEP 3

Encoding Categorical Features

Machine learning algorithms cannot directly process categorical features, so we must convert them into numerical features.

Accordingly, we are converting all categorical features in this dataset both binary and multi-class columns into numerical format.

STEP 4

Feature Scaling with StandardScaler

Our dataset contains many features, with each feature having values in different ranges. When creating a model, algorithms might inadvertently assign more weight to features with high-range values and less weight to those with low-range values. To eliminate this partiality, we must scale our features using a standard scaler. This ensures the model gives equal importance to all features.

STEP 5

Establishing a Baseline with DummyClassifier

Before training a real model, a naive reference model is built intentionally. For instance, a DummyClassifier simply predicts the most common class for every single patient, completely ignoring clinical features. This serves as a vital sanity check, establishing a minimum performance benchmark to prove whether your complex machine learning model is actually learning useful patterns.

STEP 6

Detecting Multicollinearity

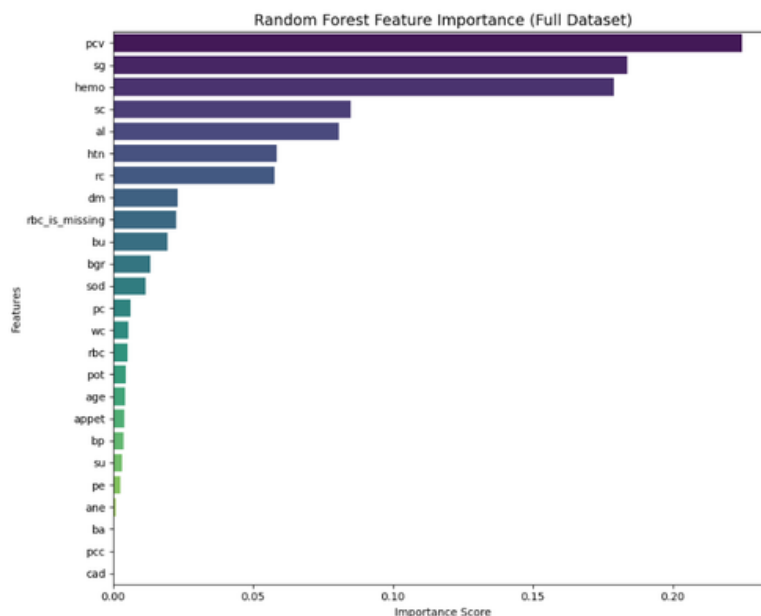
In this step, to detect multicollinearity, we use a correlation heatmap to identify highly correlated features. Additionally, we use the VIF score to pinpoint features that contribute to multicollinearity.

The analysis reveals that pcv (packed cell volume) and hemo (haemoglobin) are highly correlated — they both measure aspects of red blood cell density in the blood and therefore carry overlapping information. One of them must be dropped.

STEP 7

Measuring Feature Importance with Random Forest

In this step, we analyze the predictive power of each independent feature using the Random Forest algorithm. This will tell us how effectively our independent features can predict the target variable, and it will rank these independent features based on their predictive power.



STEP 8

Dropping the Redundant Feature

The decision to drop hemo over pcv is based on a three-way comparison across all available evidence:

Metric	pcv	hemo	Decision
Missing values	17.98%	13.25%	pcv had more missingness
VIF Score	5.13	4.69	Both high, pcv slightly worse
Feature Importance	0.2248	0.1790	pcv carries more signal

Conclusion: **hemo is dropped**. Keeping pcv preserves the stronger signal while eliminating the collinear pair. Both training and test sets are updated consistently so the pipeline stays aligned.

STEP 9

Recursive Feature Elimination with Cross-Validation (RFECV)

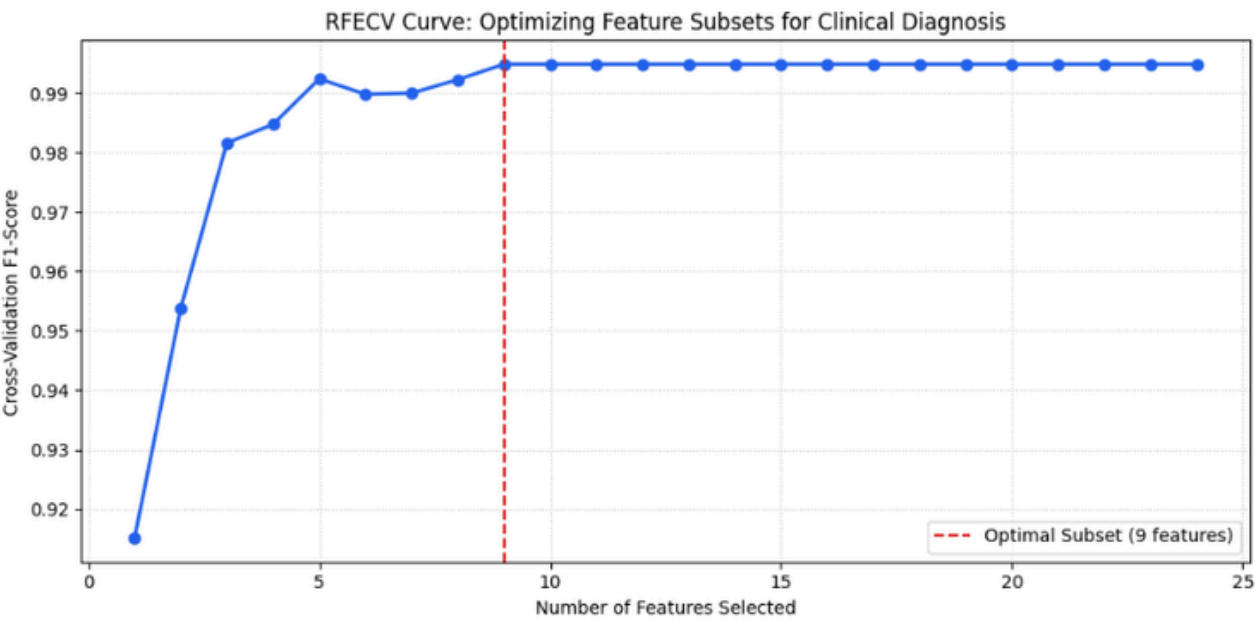
Even after removing the collinear feature, we still have 24 features. Not all of them may be genuinely useful — some could be noise that actually hurts model performance. RFECV solves this by systematically testing different subsets of features to find the optimal combination.

How RFECV works:

1. Train the model using all 24 features.
2. Rank the features by their importance score.
3. Remove the least important feature.
4. Measure the F1-Score using 5-fold cross-validation.
5. Repeat steps 2–4 until only one feature remains.
6. Keep the feature count that produced the highest F1-Score.

```
--- RFE Selection Summary ---
Original number of features : 24
Number of selected features : 9

[X] SELECTED FEATURES:
['sg', 'al', 'bgr', 'bu', 'sc', 'pcv', 'rc', 'htn', 'rbc_is_missing']
```



STEP 10

Preparing the Final Feature Matrices

In this step, we are creating a training set and a test set using the best nine features obtained through feature selection. This is necessary for model creation and testing

STEP 11

Multi-Model Performance Benchmarking

Seven different classification algorithms are trained and evaluated in a single loop to identify which model architecture best suits this dataset. Rather than guessing which model to use, and the models ranked by CV F1 score. We selected the Random Forest algorithm as a best model from the results

OPTIMIZED MODEL BENCHMARK TABLE (Ranked by CV F1)						
	Model	CV F1 (Train)	Test Accuracy	Test Precision	Test Recall	Test F1
0	Random Forest	99.49%	100.00%	100.00%	100.00%	100.00%
1	SVM (RBF)	99.23%	100.00%	100.00%	100.00%	100.00%
2	Logistic Regression	98.71%	98.75%	100.00%	98.00%	98.99%
3	Decision Tree	98.46%	96.25%	97.96%	96.00%	96.97%
4	KNN	98.21%	100.00%	100.00%	100.00%	100.00%
5	Gradient Boosting	98.20%	98.75%	100.00%	98.00%	98.99%
6	Naive Bayes	94.32%	93.75%	100.00%	90.00%	94.74%

STEP 12

Hyperparameter Tuning with GridSearchCV

In this step, we are performing a Grid Search to find the best hyperparameters for our Random Forest model. And fine-tune our model using these parameters to enhance the performance.

STEP 13

Final Model Evaluation & Diagnostic Plots

In this step, we are generating a classification report, confusion matrix, ROC curve, and other metrics. Through these, we learn how our model is performing.

STEP 14

Clinical Feature Importance Ranking

In this step, we are examining the importance our model assigns to each feature. By doing this, and with the help of domain experts, we can verify if this importance is accurate or incorrect, thereby ensuring the reliability of our model.

STEP 15

Saving the Model and Production Assets

In this step, we are using the joblib library to save our best Random Forest model. Next, we are saving our fitted StandardScaler to preserve the feature scaling. Finally, we are again using joblib to save our list of best features.

STEP 16

Pre-Deployment Safety Audit

Before deploying our model, we run a final health check on our saved files (.joblib). This block confirms the files exist, load correctly, make predictions instantly, and can handle extreme high-risk patient data safely.